S

SYNCUT · EQUIPO 2 · RUTA CRÍTICA

Autenticación, Roles, Sesiones y Auditoría

Resumen técnico del estado real del módulo y guía para exponerlo. Qué existe en base de datos, qué existe en código, qué falta y cómo contarlo en 10 minutos.

MÓDULO
Seguridad transversal (Squad 2 / Equipo 2)

INTEGRANTES
Alexander · Alan · Wallmach · Erick

STACK
Next.js 16 (App Router) · Supabase Auth · PostgreSQL + RLS

CARPETA
apps/web/app/(auth)/ · supabase/migrations/ · packages/sdk

DOCUMENTO
Resumen + guía de exposición

FECHA
11 de septiembre de 2026

01 Panorama en 60 segundos

El módulo es la **capa de seguridad transversal** de SyncUT: decide quién entra, qué rol tiene, qué filas de la base de datos puede ver y deja constancia de lo que hace. Es el bloqueador del proyecto: ningún otro squad puede consultar datos protegidos sin él.

La idea central es que **la seguridad no vive en el frontend, vive en la base de datos**. Postgres, mediante Row Level Security (RLS), filtra cada consulta según el usuario que la hace. Aunque alguien manipule el navegador, la base no devuelve filas que no le corresponden.

Las cuatro piezas

Pieza	Dónde vive	Para qué sirve
Identidad	`auth.users` (Supabase)	Credenciales, contraseña cifrada con bcrypt, confirmación de email y emisión del JWT. No la escribimos nosotros: la gestiona Supabase.
Perfil y rol	`public.profiles`	Extiende al usuario con nombre, teléfono, avatar y sobre todo `role`. Es la tabla que consultan todas las políticas de seguridad.
Permisos	`public.role_permissions`	Catálogo RBAC: qué permiso concreto tiene cada rol (`approve:justificacion`, `manage:users`…).
Trazabilidad	`public.audit_logs` y `public.session_tokens`	Bitácora inmutable de acciones y registro de dispositivos/sesiones activas.

> **LA FRASE PARA ARRANCAR LA EXPOSICIÓN**
>
> «Nuestro módulo responde tres preguntas: **¿quién eres?** (autenticación), **¿qué puedes hacer?** (roles y RLS) y **¿qué hiciste?** (auditoría). Las tres están resueltas a nivel de base de datos; la capa visual todavía va detrás.»

02 Arquitectura: las tres capas



Cómo se conecta la aplicación a Supabase

Hay un único punto de entrada, para que ningún squad cree su propio cliente:

```ts
// packages/sdk/src/client.ts
import { createBrowserClient } from '@supabase/ssr';
import type { Database } from '@plataforma/types';

export const createSupabaseBrowserClient = () => {
  return createBrowserClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
};
```

Se usa así desde cualquier componente cliente: `import { createSupabaseBrowserClient } from "@plataforma/sdk/client";`

> **PUNTO QUE IMPRESIONA EN LA EXPOSICIÓN**
>
> Los tipos de TypeScript de toda la base de datos están **autogenerados** por Supabase en `packages/types/src/database.types.ts`. Si alguien cambia una columna en SQL y no actualiza el código, **el build falla**. La base de datos y el código no pueden desincronizarse en silencio.

03 La base de datos: cuatro tablas

Todo está en `supabase/migrations/20240516000001_init_schemas.sql` (132 líneas). Es el archivo que hay que abrir si te piden «enséñame el código».

3.1 · `profiles` — el perfil y el rol

```
CREATE TABLE public.profiles (
  id          uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email       text NOT NULL,
  full_name   text NOT NULL,
  role        text NOT NULL DEFAULT 'student',
  phone       text,
  avatar_url  text,
  created_at  timestamptz DEFAULT now(),
  updated_at  timestamptz DEFAULT now(),
  CONSTRAINT role_check CHECK (role IN ('student','teacher','tutor','admin','coordinator'))
);
```

Tres detalles que conviene saber explicar:

- `id` **es a la vez clave primaria y foránea** hacia `auth.users`: relación 1 a 1. No puede existir un perfil sin usuario real.
- `ON DELETE CASCADE`: si se borra el usuario en Supabase Auth, su perfil desaparece automáticamente. Cumplimiento de «derecho al olvido» sin código extra.
- `CHECK` **sobre** `role`: es imposible insertar un rol inventado. La validación es del motor, no de la aplicación.

Índices: `idx_profiles_email` y `idx_profiles_role`, porque son las dos columnas por las que más se filtra.

3.2 · `role_permissions` — el catálogo RBAC

Tabla con `UNIQUE(role, permission)`. Permite cambiar permisos sin recompilar: un permiso es una fila, no una línea de código. Ejemplos previstos: `view:citas`, `approve:justificacion`, `manage:users`, `view:audit_logs`.

3.3 · `session_tokens` — dispositivos activos

Guarda `token` (único), `device_name`, `ip_address` (tipo `inet`), `expires_at` y `revoked_at`. El truco de diseño: **no se borra una sesión, se marca** con `revoked_at = now()`. Así queda el histórico de quién cerró sesión y cuándo. Sirve para «cerrar sesión en todos los dispositivos» y para detectar accesos desde IPs extrañas.

3.4 · `audit_logs` — la bitácora inmutable

```
CREATE TABLE public.audit_logs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid REFERENCES public.profiles(id) ON DELETE SET NULL,
  action      text NOT NULL,        -- LOGIN, SIGNUP, UPDATE, DELETE...
  table_name  text NOT NULL,        -- tabla afectada
  record_id   uuid,                    -- fila afectada
  old_values  jsonb,                   -- antes
  new_values  jsonb,                   -- después
  ip_address  inet,
  user_agent  text,
  created_at  timestamptz DEFAULT now() NOT NULL
);
```

Por qué `ON DELETE SET NULL` **y no** `CASCADE`**:** si se borra un usuario, sus registros de auditoría **sobreviven** con `user_id = NULL`. Borrar una cuenta no puede borrar la evidencia de lo que hizo. Es el detalle que distingue una bitácora de verdad de un simple log.

Por qué `old_values` **/** `new_values` **son** `jsonb`**:** una sola tabla sirve para auditar cualquier otra tabla del sistema, sin importar sus columnas.

04 Roles: el sistema RBAC

Cinco roles, definidos en cuatro lugares que deben coincidir siempre.

Rol	Qué representa
`student`	Alumno. Rol por defecto de cualquier registro nuevo. Ve y crea sus propias citas y justificaciones.
`teacher`	Docente. Pasa lista, aprueba justificaciones, genera notificaciones.
`tutor`	Tutor académico. Acompañamiento y agenda de tutorías.
`coordinator`	Coordinador. Gestiona usuarios y grupos, consulta auditoría.
`admin`	Administrador. Acceso total, incluido el Panel de Gobernanza.

Dónde está declarado cada rol (la «fuente de verdad» repetida)

Archivo	Constante	Función
`supabase/migrations/…_init_schemas.sql`	`CONSTRAINT role_check`	Autoridad final. Nada entra si no está aquí.
`packages/sdk/src/types.ts`	`type UserRole`	Tipado en TypeScript.
`packages/config/src/index.ts`	`CORE_ROLES`	Configuración compartida entre squads.
`packages/shared/src/validators.ts`	`VALID_ROLES`	Validación en tiempo de ejecución.

> **RIESGO CONOCIDO — DILO TÚ ANTES DE QUE LO PREGUNTEN**
>
> La lista de roles está duplicada en cuatro sitios. Si mañana se añade un rol `guest`, hay que tocar los cuatro o el sistema se rompe de forma silenciosa. **Mejora propuesta:** convertirlo en un `ENUM` de Postgres y que TypeScript lo derive del tipo autogenerado, con una sola fuente de verdad.

Asignación automática del rol: el trigger

Nadie asigna el rol a mano. Cuando Supabase crea un usuario, un trigger crea el perfil:

```sql
-- supabase/migrations/20260523000001_sync_auth_profiles.sql
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, role)
  VALUES (
    new.id,
    new.email,
    COALESCE(new.raw_user_meta_data->>'full_name', 'Nuevo Usuario'),
    COALESCE(new.raw_user_meta_data->>'role', 'student')
  );
  RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

Dos palabras clave que te pueden preguntar:

- `SECURITY DEFINER`: la función se ejecuta con los privilegios de quien la creó, no de quien la dispara. Es lo que permite escribir en `profiles` aunque el usuario nuevo todavía no tenga permisos.
- `COALESCE`: toma el primer valor no nulo. Si el registro no manda rol, queda `'student'`. **El privilegio mínimo es el valor por defecto.**

El administrador maestro

La migración `20260606000001_set_master_admin.sql` reescribe el trigger para que un correo concreto (`jassiel.rr1502@gmail.com`) reciba siempre `role = 'admin'`, y además hace un `UPDATE` por si el perfil ya existía. Garantiza que el proyecto nunca se quede sin administrador.

> **OBSERVACIÓN HONESTA SOBRE ESTA DECISIÓN**
>
> Es un correo **quemado en el código SQL**. Funciona para la etapa académica del proyecto, pero en producción el administrador debería promoverse desde una tabla de configuración o una variable de entorno, no desde una migración versionada en el repositorio.

05 RLS: la seguridad que vive en la base de datos

Row Level Security es el corazón del módulo y el mejor argumento técnico de la exposición. Es un filtro `WHERE` invisible que Postgres añade a **toda** consulta, venga de donde venga.

Se activa tabla por tabla:

```sql
ALTER TABLE public.profiles         ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.audit_logs       ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.session_tokens   ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.role_permissions ENABLE ROW LEVEL SECURITY;
```

La función clave es `auth.uid()`: devuelve el UUID del usuario extraído del JWT firmado. No se puede falsificar desde el navegador, porque la firma se verifica en el servidor.

Las seis políticas implementadas

Política	Operación	Regla en una línea
`profiles_self_view`	SELECT	Ves tu propio perfil; los admin ven todos.
`profiles_update_own`	UPDATE	Editas tu propio perfil; los admin editan cualquiera.
`audit_logs_view`	SELECT	Ves tus propios eventos; los admin ven el histórico completo.
`audit_logs_no_delete`	DELETE	`USING (false)` — nadie borra nunca. Inmutabilidad real.
`session_tokens_view`	SELECT	Ves tus dispositivos; los admin ven todos.
`role_permissions_view`	SELECT	`USING (true)` — el catálogo de permisos es público para usuarios autenticados.

La política que hay que leer en voz alta

```
CREATE POLICY "audit_logs_view"
  ON public.audit_logs FOR SELECT TO authenticated
  USING (
    user_id = auth.uid()                      -- tus propios eventos
    OR EXISTS (
      SELECT 1 FROM public.profiles
      WHERE id = auth.uid() AND role = 'admin'   -- o eres admin
    )
  );

CREATE POLICY "audit_logs_no_delete"
  ON public.audit_logs FOR DELETE TO authenticated
  USING (false);   -- ni el admin puede borrar la evidencia
```

Qué demostrar: un alumno que ejecute `SELECT * FROM audit_logs` no recibe un error de permisos: recibe **cero filas**. La base de datos no revela siquiera que existan otros registros.

> **DIFERENCIA DE `USING` Y `WITH CHECK`**
>
> `USING` filtra las filas que **puedes leer o tocar**. `WITH CHECK` valida las filas que **intentas escribir**. En `profiles_update_own` están los dos, para que nadie lea su propia fila y la convierta en la de otro usuario.

> **HUECO DE SEGURIDAD DETECTADO — MENCIONARLO SUMA PUNTOS**
>
> Las políticas `profiles_self_view` y `profiles_update_own` usan `auth.uid() = id OR role = 'admin'`. Ese `role` es **el de la fila que se está evaluando**, no el del usuario que consulta. Efecto real: cualquier usuario autenticado puede leer y modificar los perfiles cuyo rol sea `admin`. Es decir, **un alumno podría editar el perfil del administrador**.
>
> Además, `profiles_update_own` no impide que un usuario se cambie su propio `role` a `admin` — el documento de diseño lo da por protegido, pero la política no lo implementa.
>
> **Corrección:** usar la subconsulta `EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin')`, igual que en `audit_logs_view`, y añadir un `WITH CHECK` que congele la columna `role` salvo para administradores.

06 Registro y login: diseño frente a implementación actual

Aquí está el punto más importante de honestidad de la exposición: el **diseño** del flujo está completo y documentado, pero el **código actual** de las pantallas es todavía una maqueta funcional que no habla con Supabase.

6.1 · Flujo de registro diseñado

1. El usuario llena nombre, correo y contraseña. Validación en el navegador.
2. Se llama a `supabase.auth.signUp({ email, password, options: { data: { full_name } } })`.
3. Supabase crea la fila en `auth.users`, cifra la contraseña con bcrypt y envía correo de confirmación.
4. El trigger `handle_new_user()` dispara y crea el perfil con rol `student`.
5. Se registra el evento `SIGNUP` en `audit_logs`.
6. El usuario confirma su correo y ya puede iniciar sesión.

6.2 · Flujo de login diseñado

1. `supabase.auth.signInWithPassword({ email, password })` valida contra `auth.users`.
2. Si es correcto: se emite el JWT, se inserta la fila en `session_tokens` (dispositivo, IP, expiración) y se guarda una cookie `HttpOnly`.
3. Se registra `LOGIN` en `audit_logs`.
4. Si falla: se registra `LOGIN_FAILED` con IP y motivo; a partir de tres intentos en 15 minutos se bloquea temporalmente.
5. En cada petición posterior, el middleware valida el JWT y RLS filtra los datos.

6.3 · Lo que hace el código hoy

`apps/web/app/(auth)/login/page.tsx` — componente `"use client"` con formulario, mostrar/ocultar contraseña, «recordar sesión» y manejo de errores. La validación es una maqueta:

```
// Mock validation to match the mockup error state
if (password !== "hunter2") {
  setErrorMsg("Contraseña incorrecta. Por favor intente de nuevo.");
  return;
}

window.localStorage.setItem("syncut_beta_session", JSON.stringify({
  email: cleanUsername,
  role: cleanUsername.toLowerCase().includes("admin") ? "admin" : "student",
  loggedAt: new Date().toISOString(),
}));

router.push("/dashboard");
```

`apps/web/app/(auth)/signup/page.tsx` — valida campos obligatorios, longitud mínima de 8 caracteres, coincidencia de contraseñas y aceptación de términos; incluye un medidor visual de fuerza. Después guarda el usuario en `localStorage["syncut_beta_users"]` y redirige al login.

> **CÓMO DECIRLO SIN QUE SUENE MAL**
>
> «Las pantallas de login y registro están construidas contra la maqueta de diseño y usan `localStorage` como sesión temporal, para que los otros cinco squads pudieran navegar y desarrollar sus módulos sin esperarnos. La contraseña de demostración es literal y la sesión vive solo en el navegador: **no es un mecanismo de seguridad, es un andamio**. El siguiente paso es sustituir esas dos funciones por `signUp` y `signInWithPassword`; la base de datos ya está lista para recibirlas.»

> **DETALLE TÉCNICO QUE CONVIENE TENER CLARO**
>
> En la maqueta, el rol se decide con `email.includes("admin")`, en el navegador. Eso significa que hoy cualquiera podría escribir `admin@loquesea.com` y obtener rol admin **en el estado local**. No otorga acceso a datos reales, porque RLS sigue vigente en Supabase y el JWT nunca se emite; el efecto se limita a la interfaz.

Los validadores «de verdad» ya existen y están listos para conectarse, en `packages/shared/src/validators.ts`:

```
validateEmail(email)     // formato + rechazo de patrones de inyección ( ; -- /* )
validatePassword(pass)   // 8+ caracteres, mayúscula, minúscula y número
validateRole(role)       // pertenencia a VALID_ROLES
```

07 Protección de rutas y sesión

Hoy la protección ocurre en **tres puntos del cliente**, y falta el cuarto (el servidor).

1 · Redirección de entrada

`apps/web/app/page.tsx`

Si hay `syncut_beta_session` manda a `/dashboard`; si no, a `/login`.

2 · Menú por rol

`apps/web/app/(dashboard)/layout.tsx`

Lee la sesión, calcula `isMasterAdmin` y filtra el enlace «Panel Gobernanza» del menú. También implementa el logout: borra la clave y redirige.

3 · Verificación real de rol

`apps/web/app/(dashboard)/admin/page.tsx`

Es el único lugar que consulta Supabase de verdad: `auth.getUser()` y luego el `role` en `profiles`. Si no es `admin`, pinta «Acceso Restringido».

4 · Middleware PENDIENTE

`apps/web/middleware.ts`

No existe todavía. Es la pieza que debe validar el JWT y refrescar la cookie **antes** de renderizar cualquier ruta protegida.

La verificación en cascada del panel de administración

Vale la pena explicarla porque muestra pensamiento defensivo: tiene tres niveles de respaldo.

```
// 1) ¿Hay sesión local? Si no, acceso denegado.
const localSession = window.localStorage.getItem("syncut_beta_session");

// 2) ¿Hay variables de entorno de Supabase? Entonces manda la base de datos:
const { data: { user } } = await supabase.auth.getUser();
const { data: profile } = await supabase
  .from("profiles").select("role").eq("id", user.id).single();
if (profile?.role === "admin") return autorizado;

//    2b) respaldo: buscar el perfil por el correo de la sesión local
// 3) Sin variables de entorno (modo maqueta offline): validación local
const isMockAdmin = email === "jassiel.rr1502@gmail.com"
  || parsed?.role === "admin" || email?.includes("admin");
```

La jerarquía correcta está bien planteada: si la base de datos responde, la base de datos decide; el modo local solo actúa cuando no hay conexión configurada.

08 Logs y auditoría

La auditoría está **diseñada al completo, leída en el panel ejecutivo y todavía no escrita** por la aplicación. Es el estado exacto que hay que comunicar.

Eventos previstos

Autenticación	Control de acceso	Datos

`LOGIN`, `LOGOUT`, `LOGIN_FAILED`, `SIGNUP`, `PASSWORD_CHANGE`, `PASSWORD_RESET`, `EMAIL_CHANGE`, `EMAIL_CONFIRMED`, `SESSION_REVOKED`, `BAN_USER`

`ROLE_UPDATED`, `PERMISSION_GRANTED`, `PERMISSION_REVOKED`, `ACCESS_DENIED`

`PROFILE_UPDATE`, `CITA_CREATE/UPDATE/DELETE`, `JUSTIFICACION_CREATE/UPDATE`, `NOTIFICACION_SEND`

La función de escritura (especificada, aún no creada)

```
// apps/web/app/(auth)/lib/auditLog.ts  ← archivo por implementar
export async function logAudit(entry: AuditLogEntry) {
  const { data, error } = await supabase.from('audit_logs').insert({
    user_id: entry.userId,     action:     entry.action,
    table_name: entry.tableName, record_id: entry.recordId,
    old_values: entry.oldValues, new_values: entry.newValues,
    ip_address: entry.ipAddress, user_agent: entry.userAgent,
  });
  if (error) console.error('Audit log failed:', error);
  // No se lanza excepción: auditar nunca debe romper la operación del usuario
  return data;
}
```

Esa última decisión de diseño es importante y conviene subrayarla: **si falla la auditoría, la operación del usuario continúa**. Se registra el problema, pero no se rompe la experiencia.

Lo que sí está funcionando: la lectura en tiempo real

El Panel de Gobernanza consume `audit_logs` y los convierte en un feed de actividad.

`apps/web/components/modules/executive-dashboard/live-data.ts` — consulta inicial:

```
await supabase
  .from("audit_logs")
  .select("user_id, action, table_name, created_at", { count: "exact" })
  .order("created_at", { ascending: false })
  .limit(30);
```

`…/executive-dashboard-page.tsx` — suscripción a Supabase Realtime, que empuja cada nuevo registro al feed sin recargar la página:

```
supabase.channel("executive-audit-stream")
  .on("postgres_changes",
     { event: "INSERT", schema: "public", table: "audit_logs" },
     (payload) => { /* añade la actividad al feed en vivo */ })
  .subscribe();
```

Hay además funciones auxiliares que enriquecen cada evento: `inferModule()` deduce a qué módulo pertenece según la tabla, `inferAction()` clasifica el tipo de acción e `inferImpact()` asigna un nivel de impacto (Alto / Medio / Bajo). Los datos se pueden exportar a CSV con `toCsvReport()`.

> **EL RESUMEN DE UNA FRASE**
>
> «La tubería de auditoría está construida de la mitad hacia adelante: la tabla existe, es inmutable, se lee y se transmite en vivo al panel ejecutivo. Lo que falta es el emisor — `logAudit()` — conectado a los puntos críticos. Es trabajo de conexión, no de arquitectura.»

Todo el sistema degrada con elegancia: si no hay `NEXT_PUBLIC_SUPABASE_URL` ni `NEXT_PUBLIC_SUPABASE_ANON_KEY`, las funciones devuelven datos de respaldo y marcan `hasLiveSource: false` en lugar de fallar.

09 Estado real del módulo

Tabla de referencia. Si solo memorizas una página del documento, que sea esta.

Componente	Estado	Detalle
Esquema de BD (4 tablas)	LISTO	Migración inicial con índices y constraints.
Políticas RLS	PARCIAL	6 políticas activas; falta corregir la comparación de rol en `profiles`.
Trigger `handle_new_user()`	LISTO	Crea perfil automáticamente con rol por defecto.
Admin maestro	LISTO	Garantizado por migración.
Cliente Supabase (SDK)	LISTO	`createSupabaseBrowserClient()` centralizado.
Tipos autogenerados	LISTO	`packages/types/src/database.types.ts`.
Validadores	LISTO	Email, contraseña y rol; sin conectar a los formularios.
UI de login y registro	LISTO	Accesible, con estados de error y medidor de fuerza.
Lógica de login / registro	MAQUETA	Usa `localStorage`; no llama a Supabase Auth.
Escritura de `audit_logs`	PENDIENTE	`logAudit()` especificado, no implementado.
Uso de `session_tokens`	PENDIENTE	Tabla creada, sin inserción ni revocación.
Poblado de `role_permissions`	PENDIENTE	Tabla vacía; permisos aún definidos solo en el documento.
`middleware.ts`	PENDIENTE	No hay validación de sesión del lado del servidor.
Recuperación de contraseña	PENDIENTE	El enlace existe en la UI y apunta a `#`.
MFA / 2FA	PENDIENTE	Planificado en el Sprint 1 del MASTER.
Lectura de auditoría	LISTO	Consulta + Realtime + exportación CSV en el panel ejecutivo.

Los cinco siguientes pasos, en orden

1. **Corregir las políticas RLS de `profiles`** con la subconsulta `EXISTS` y un `WITH CHECK` que proteja la columna `role`. Es seguridad, va primero.
2. **Sustituir la maqueta por Supabase Auth** en las dos pantallas, usando los validadores que ya existen.
3. **Crear `apps/web/middleware.ts`** para validar la sesión en el servidor antes de renderizar rutas protegidas, y quitar la dependencia de `localStorage`.
4. **Implementar `logAudit()`** y engancharlo a login, logout, registro, cambio de rol y cambio de contraseña. El panel se llena solo.
5. **Poblar `role_permissions`** y construir un *helper* `can(permission)` que los demás squads consuman.

10 Guía de exposición · guion de 10 minutos

0:00–1:00 — **Encuadre: por qué este módulo es la ruta crítica**

> *«Somos el Equipo 2. Nuestro módulo no tiene pantalla propia que lucir, pero sin él ningún otro squad puede leer un dato protegido. Resolvemos tres preguntas: quién eres, qué puedes hacer y qué hiciste.»*

1:00–2:30 — **Arquitectura de tres capas**

Muestra el diagrama de la sección 2. Insiste en la idea fuerza: la seguridad vive en la base de datos, no en el navegador.

> *«Si alguien abre las herramientas de desarrollo y modifica el frontend, no consigue nada: Postgres filtra las filas antes de que salgan del servidor.»*

2:30–4:30 — **Las cuatro tablas**

Abre la migración inicial. Destaca tres decisiones de diseño, no las nueve columnas: `ON DELETE CASCADE` en perfiles, `ON DELETE SET NULL` en auditoría y el `CHECK` de roles.

> *«Fíjense en el contraste: al borrar un usuario, su perfil desaparece pero su auditoría sobrevive. Borrar una cuenta no puede borrar la evidencia.»*

4:30–6:00 — **Roles y el trigger automático**

Explica los cinco roles y lee `handle_new_user()`. Menciona `SECURITY DEFINER` y el privilegio mínimo por defecto.

> *«Nadie asigna roles a mano. Al registrarse, la base de datos crea el perfil sola, siempre con el rol de menor privilegio.»*

6:00–7:30 — **RLS en vivo — el momento fuerte**

Lee `audit_logs_view` y `audit_logs_no_delete`. Si puedes, demuéstralo en el panel de Supabase.

> *«`USING (false)`: ni el administrador puede borrar un registro de auditoría. Eso es inmutabilidad garantizada por el motor, no por una promesa del código.»*

7:30–9:00 — **Demostración: login → dashboard → panel restringido**

Entra con un usuario normal, enseña que «Panel Gobernanza» no aparece en el menú; intenta entrar por URL y muestra la pantalla «Acceso Restringido». Luego entra como admin y muestra el feed de auditoría.

9:00–10:00 — **Estado real y siguientes pasos**

Proyecta la tabla de la sección 9 y cierra con los cinco pasos. Declara tú mismo la deuda técnica.

> *«Los cimientos están terminados y son sólidos. Lo que falta no es arquitectura, es conexión: reemplazar la maqueta de las dos pantallas por las llamadas reales y encender el emisor de auditoría.»*

Cuatro cosas que no deben faltar

- Decir la palabra **RLS** y explicarla en una frase entendible.
- Enseñar **código real** en pantalla, no solo diagramas: la migración y el trigger.
- Declarar la **deuda técnica** antes de que te pregunten. Genera confianza.
- Cerrar conectando con los otros squads: «cuando encendamos `logAudit()`, el panel del Squad 5 se llena solo».

11 Preguntas que te van a hacer (con respuesta)

¿Dónde se guardan las contraseñas?

Nunca en nuestras tablas. Viven cifradas con bcrypt en `auth.users`, que administra Supabase. Nuestra tabla `profiles` no tiene ni puede tener una columna de contraseña.

¿Qué pasa si alguien modifica el JavaScript del navegador para ponerse rol admin?

Cambiaría lo que ve en pantalla, nada más. El rol que importa se lee de `profiles` y las políticas RLS lo evalúan del lado del servidor usando `auth.uid()`, que sale del JWT firmado. Sin JWT válido, la base devuelve cero filas.

¿Por qué duplicar el correo en `profiles` si ya está en `auth.users`?

Es denormalización deliberada. `auth.users` está en un esquema interno de Supabase al que no se puede hacer `JOIN` libremente desde el cliente. Tener el correo en `profiles` evita una llamada extra en cada consulta y permite indexarlo.

¿Cómo garantizan que un log no se pueda alterar?

Con la política `audit_logs_no_delete` que devuelve `USING (false)`: ningún rol autenticado puede borrar. Tampoco existe política de `UPDATE`, y en RLS lo que no está permitido explícitamente está prohibido. Además `created_at` lo pone el servidor, no el cliente.

¿Y si borran un usuario, se pierde su auditoría?

No. La clave foránea es `ON DELETE SET NULL`: los eventos permanecen con `user_id` nulo. Es una decisión de diseño consciente y es lo que exige cualquier norma de cumplimiento.

¿El login ya funciona contra la base de datos?

Todavía no. Las pantallas están terminadas y el esquema está listo, pero la validación actual es una maqueta con `localStorage` que construimos para desbloquear a los otros cinco squads. La sustitución son dos llamadas: `signUp` y `signInWithPassword`.

¿Qué es `SECURITY DEFINER` y por qué lo usan?

Hace que la función se ejecute con los privilegios de quien la creó. Sin eso, el trigger no podría insertar en `profiles` en el instante en que nace el usuario, porque todavía no tiene permisos propios.

¿Cuál es la diferencia entre roles y permisos?

El rol es la etiqueta del usuario y vive en `profiles.role`. El permiso es la acción concreta y vive en `role_permissions`. Separarlos permite cambiar qué puede hacer un docente sin tocar el código ni migrar usuarios.

¿Qué falta y cuánto cuesta?

Cinco cosas: corregir dos políticas RLS, conectar Supabase Auth a las dos pantallas, crear el middleware, implementar `logAudit()` y poblar `role_permissions`. Todo es trabajo de integración sobre cimientos que ya existen; no hay que rediseñar nada.

¿Por qué no usaron NextAuth / JWT propio?

Porque Supabase Auth ya emite un JWT que Postgres entiende de forma nativa mediante `auth.uid()`. Esa integración es la que permite que RLS funcione. Con un sistema de autenticación externo tendríamos que replicar toda la autorización en la capa de aplicación, que es justo lo que queríamos evitar.

12 Chuleta de archivos

Ten esta página abierta durante la exposición. Son los archivos que vas a querer mostrar.

Archivo	Qué enseñar en él
`supabase/migrations/20240516000001_init_schemas.sql`	Las 4 tablas, los índices, el `CHECK` de roles y las 6 políticas RLS. **El archivo principal.**
`supabase/migrations/20260523000001_sync_auth_profiles.sql`	El trigger `handle_new_user()` con `SECURITY DEFINER`.
`supabase/migrations/20260606000001_set_master_admin.sql`	Garantía de administrador maestro mediante `CASE` en el trigger.
`supabase/migrations/20260605000003_complete_project_schema.sql`	Cómo el resto de módulos reutiliza `profiles` y el patrón `EXISTS … role = 'admin'`.
`apps/web/app/(auth)/AUTH_DATABASE.md`	La especificación completa del módulo: flujos, eventos y checklist por sprint.
`apps/web/app/(auth)/login/page.tsx`	La UI y la validación de maqueta que hay que sustituir.
`apps/web/app/(auth)/signup/page.tsx`	Validaciones de formulario y medidor de fuerza de contraseña.
`apps/web/app/(dashboard)/layout.tsx`	Menú filtrado por rol y función de cierre de sesión.
`apps/web/app/(dashboard)/admin/page.tsx`	La verificación de rol en cascada contra Supabase. El código más «real» del módulo.
`packages/sdk/src/client.ts`	Punto único de conexión a Supabase.
`packages/shared/src/validators.ts`	Validadores de email, contraseña y rol, con rechazo de patrones de inyección.
`…/executive-dashboard/live-data.ts`	Consulta de `audit_logs`, enriquecimiento de eventos y exportación a CSV.
`…/executive-dashboard/executive-dashboard-page.tsx`	Suscripción Realtime al canal `executive-audit-stream`.

Glosario de bolsillo

Término	Definición en una línea
JWT	Credencial firmada que el navegador envía en cada petición; no se puede falsificar sin la clave del servidor.
`auth.uid()`	Función de Postgres que extrae el UUID del usuario desde el JWT. La base de cada política.
RLS	Row Level Security: filtro automático de filas aplicado por Postgres en toda consulta.
RBAC	Control de acceso basado en roles: se asignan permisos a roles, no a personas.

Trigger	Función que la base de datos ejecuta sola cuando ocurre un evento (aquí, un `INSERT` en `auth.users`).
`USING` / `WITH CHECK`	Filtra lo que puedes leer / valida lo que puedes escribir.
bcrypt	Algoritmo de hash lento diseñado para contraseñas; hace inviable la fuerza bruta.
Inmutabilidad	Propiedad de un registro que no admite modificación ni borrado. Aquí se logra con `USING (false)`.
Realtime	Canal de Supabase que empuja cada `INSERT` al navegador sin recargar.
`anon key`	Clave pública del cliente. No otorga privilegios por sí sola: RLS sigue mandando.

> **FRASE DE CIERRE**
>
> «El módulo de autenticación de SyncUT apuesta por poner la seguridad donde están los datos. El esquema, los roles, las políticas y la bitácora inmutable ya están en producción en la base de datos. Lo que queda por delante es conectar la interfaz a esos cimientos —y esos cimientos, a diferencia de una pantalla, son la parte que no se puede improvisar después.»